

Report: VAT (Visual Assessment of Tendency) Clustering

GROUP MEMBERS(No:6) :

1. Sravana Sakthidharan
2. Anu Neduvely Ashokan

Introduction

The Visual Assessment of Tendency (VAT) algorithm is a technique used to visually assess the tendency of a dataset to form clusters. This algorithm works by arranging a dissimilarity matrix (distance matrix) in a way that highlights any structure in the data. VAT uses the matrix ordering to create a visual representation of the clustering tendency, where patterns such as clusters or groups are made visible through ordered matrix plots.

In this report, the VAT algorithm has been implemented in a modular way, separating the core algorithm, testing, and main execution logic into different Python files for clarity and maintainability.

Algorithm Overview

The following steps are performed by the VAT algorithm:

```
class VAT:
    """Visual Assessment of Tendency (VAT) implementation matching paper format"""

    def __init__(self, normalize=True, colormap='gray_r', n_samples_max=5000):
        self.normalize = normalize
        self.n_samples_max = n_samples_max
        self.cmap = plt.cm.gray_r if colormap == 'gray_r' else LinearSegmentedColormap.from_list('vat_cmap', ['black', 'white'], N=256)

    def fit(self, data):
        if isinstance(data, pd.DataFrame):
            data = self._preprocess_data(data)

        if len(data) > self.n_samples_max:
            data = data[np.random.choice(len(data), self.n_samples_max, replace=False)]

        # Store original data for visualization
        self.original_data = data.copy()

        # Compute dissimilarity matrix
        if data.shape[1] > 2 and self._is_nonlinear(data):
            self.R_ = self._geodesic_distance(data)
        else:
            self.R_ = squareform(pdist(data, 'euclidean'))

        if self.normalize:
            self.R_ = (self.R_ - self.R_.min()) / (self.R_.max() - self.R_.min())

        # VAT ordering
        self.order_ = self._vat_ordering(self.R_)
        self.R_ordered_ = self.R_[np.ix(self.order_, self.order_)]
        return self
```

The VAT class uses a methodical way in identifying clustering patterns in data sets by executing the Visual Assessment of Tendency (VAT) algorithm. Performance computation is user specifiable as being accomplished using a few samples, assigning a colormap to use for visualization, and normalizing the distance matrix during initialization. When applied with input data, the fit function first deals with missing values and, as required, one-hot encodes categorical variables. To ensure the computation speed, random sampling is used if the dataset is too big. The program then calculates a geodesic distance for non-linear manifolds or a Euclidean distance dissimilarity matrix for linear data. The matrix values are normalized to the range 0-1 to ensure the distances are equal if normalization is enabled. The dissimilarity matrix is then reordered by VAT ordering to identify promising clusters, and the ordered matrix is remembered to be displayed. The VAT algorithm is a good exploratory data analysis tool because of its systematic use, which means that it can efficiently discover clustering structures.

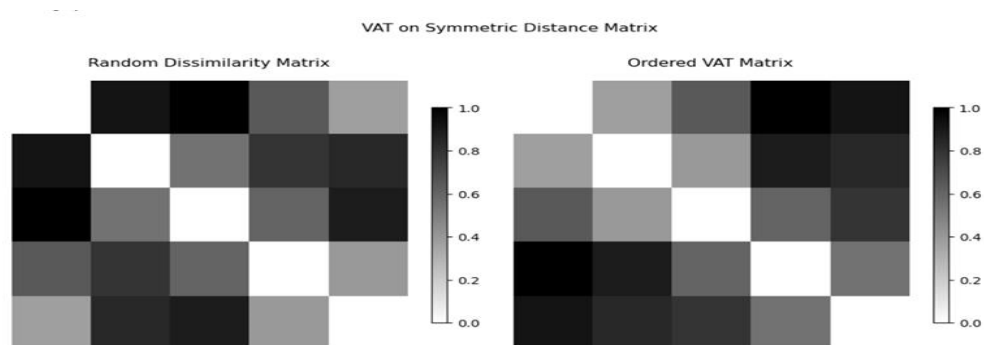
Modular Code Structure

The VAT algorithm has been refactored into three distinct Python files:

1. **algorithm.ipynb**: Contains the core implementation of the VAT algorithm, including methods for data preprocessing, distance calculation, VAT ordering, and plotting.
2. **testing.ipynb**: Includes test functions for running VAT on various datasets, such as symmetric matrices, circular data, IRIS dataset, mixed data, and large datasets. It also contains a method for testing custom CSV files.
3. **main.ipynb**: This serves as the main entry point for running the tests. It imports functions from testing.py and executes the tests.
4. **Vat_Clustering.ipynb**: Includes complete code containing all the above files.

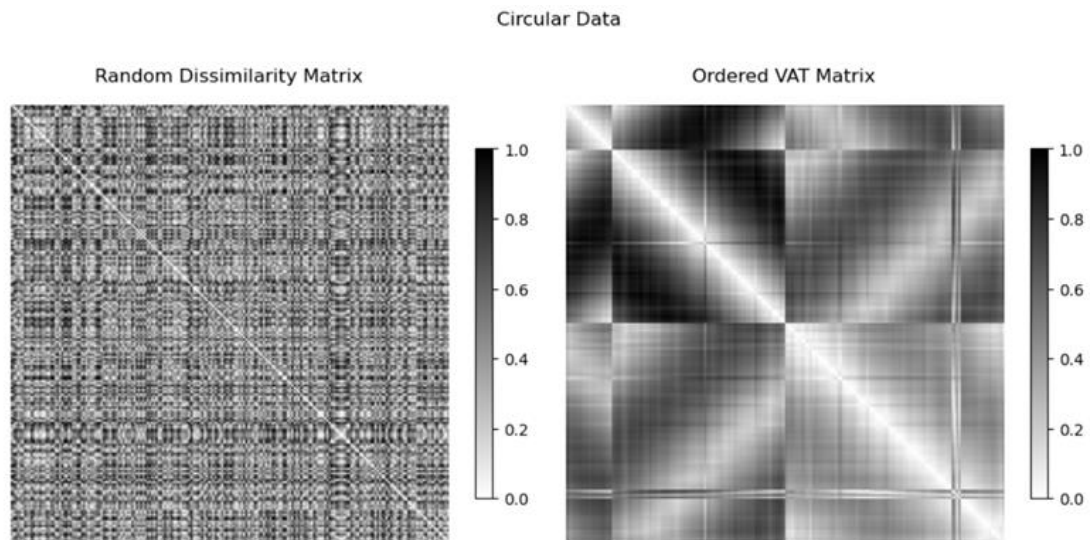
Code Execution and Results

Test 1: Symmetric Distance Matrix



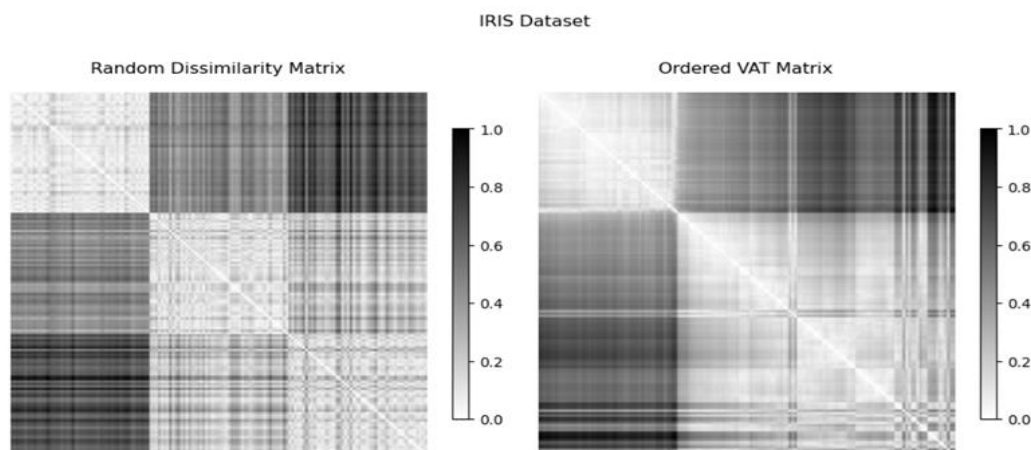
- An example of random symmetric dissimilarity matrix is generated, then VAT is computed on the resulting matrix. It gives two successive plots:
 - The first one is the random dissimilarity matrix.
 - The second is the ordered dissimilarity matrix following VAT, showing the intrinsic cluster structure.

Test 2: Circular Data



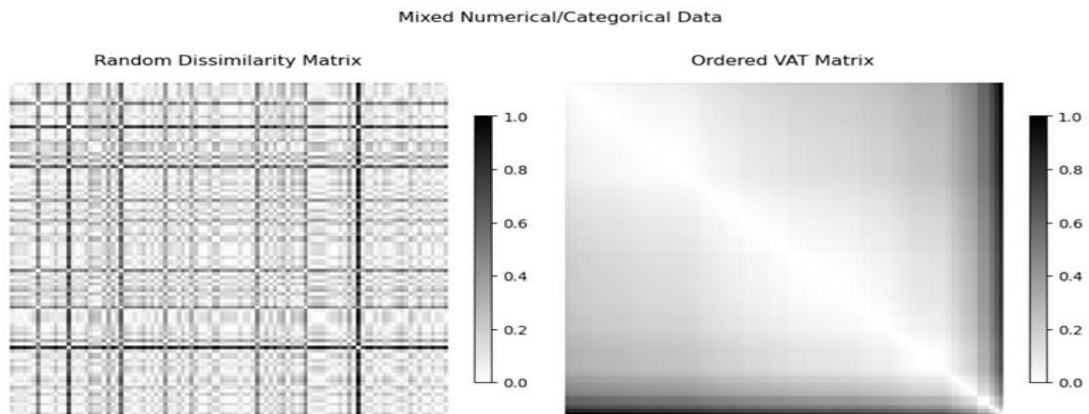
- The data set has 200 points with some noise and a circular shape.
- Once the VAT has been applied, the requested matrix produces a noticeable pattern that traces out the circular organization of the data.

Test 3: IRIS Dataset



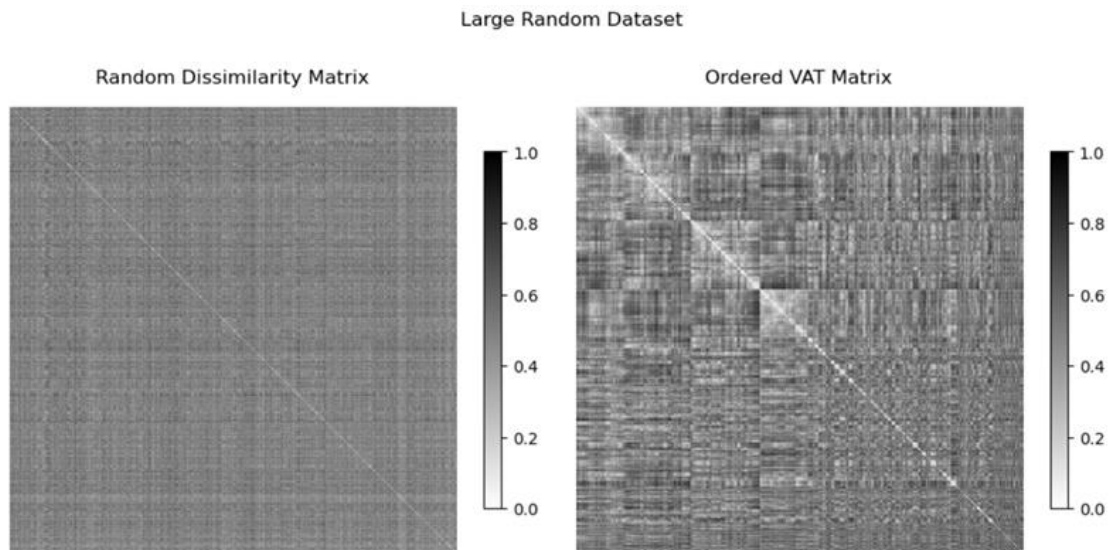
- IRIS data, a very ordinary data set of 150 cases and 4 attributes, is used using VAT.
- The formatted VAT matrix easily displays the groupings for all three species of the IRIS dataset.

Test 4: Mixed Data



- A simulated dataset consisting of a mix of numerical and categorical variables (e.g., gender, education, income, and age) is used.
- The VAT matrix correctly visualizes the structures in the pre-processed mixed data (i.e., the coded categorical variables).

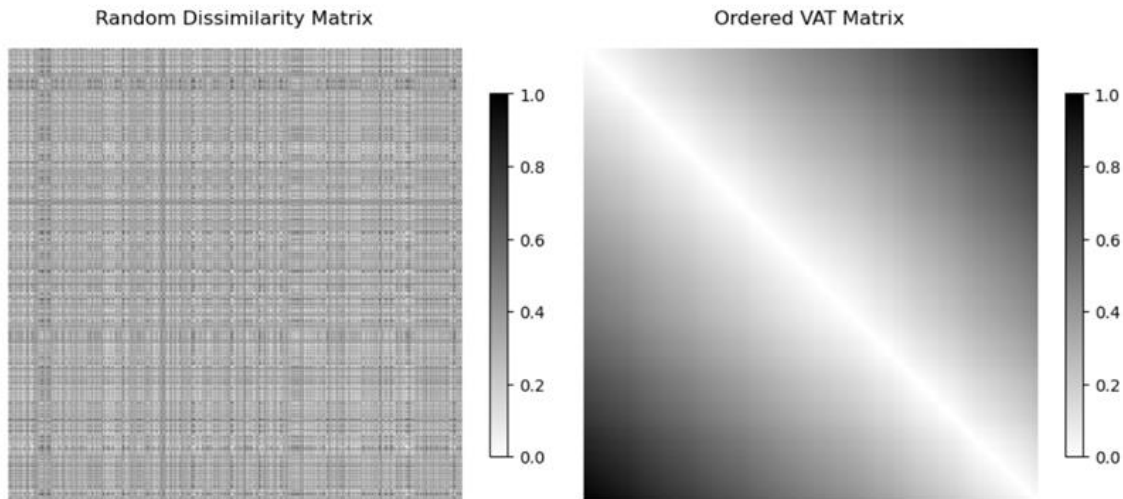
Test 5: Large Dataset



- We create a big random dataset with 1000 samples and 5 features.
- VAT works on the dataset and generates a matrix that does not show any visible structure, as expected with random data

Test 6: Custom CSV Dataset

Custom Dataset: Predict Hair Fall



A specific CSV file (Predict Hair Fall is employed here) is processed by VAT. The CSV is read, missing values are imputed, and VAT clustering is performed. A graph is created displaying the VAT outcomes for the chosen dataset.

Custom CSV Test Example

```
def test_custom_csv(csv_path):  
    """Test the custom CSV dataset and apply VAT clustering"""  
    try:  
        df = pd.read_csv(csv_path)  
  
        # Keep only numeric columns  
        numeric_df = df.select_dtypes(include=[np.number])  
  
        if numeric_df.empty:  
            raise ValueError("No numeric columns found in the CSV.")  
  
        # Impute missing values with column means  
        imputed_data = numeric_df.fillna(numeric_df.mean()).values  
  
        if imputed_data.shape[0] < 2:  
            raise ValueError("CSV must contain at least 2 rows of numeric data")  
  
        # Run VAT  
        vat = VAT()  
        vat.fit(imputed_data)  
  
        # Plot  
        dataset_name = os.path.splitext(os.path.basename(csv_path))[0]  
        vat.plot_paper_style(title=f"Custom Dataset: {dataset_name}")  
        plt.show()  
  
    except Exception as e:  
        print(f"❌ Error processing {csv_path}: {str(e)}")  
        print("🔍 Please check the CSV file:")  
        print("- It should contain at least 2 rows of numeric data")  
        print("- Missing values are now automatically handled")  
        print("- Still having issues? Consider manually cleaning data")
```

This function allows you to pass the path of a CSV file, which is processed by VAT. The data is imputed (if necessary), and VAT is applied to visualize the clustering tendency.

Conclusion

Structuring the VAT clustering algorithm in modular form improves readability, maintainability, and scalability of code. The algorithm is tested and measured on different datasets, both real and synthetic, like the IRIS dataset. Additionally, the in-house CSV test function provides a simple method to test user-provided datasets. Preprocessing steps that may be performed in the future include outlier handling, feature selection, and performance improvement for larger datasets.